# NeuroStream

## Backend technical guide

From recorded EEG to a real-time seizure alert: the data, the preprocessing, the model, the validation, the streaming pipeline, and the open questions about scale.

| 97.6% | 95.1% | 93.2% | 0.998 |
|---|---|---|---|
| accuracy | sensitivity | precision | ROC AUC |

Scored once on 75 held-out recordings (1,725 one-second epochs). Benchmark data, not clinical evidence.

Research prototype. Not a medical device and not intended for clinical use.
Repository: github.com/GiliCohen1/NeuroStream.io · Document generated 2026-11-03.

# Contents

**1. Executive summary** 4

**2. System overview** 5

2.1 The three message formats . . . 6

**3. The data** 6

3.1 What it is . . . 6

3.2 How it is stored and used . . . 7

3.3 Why this dataset . . . 7

3.4 Limitations of the data (important) . . . 7

**4. Preprocessing** 7

4.1 Step 1: validation . . . 7

4.2 Step 2: sliding-window normalization . . . 7

4.3 Step 3: band power by FFT . . . 8

**5. The model** 9

5.1 Architecture . . . 9

5.2 Design decisions . . . 9

**6. Training** 10

6.1 Procedure . . . 10

6.2 Why overlapping windows only for training . . . 11

6.3 Reproducibility . . . 11

**7. Testing and validation of the model** 11

7.1 The metrics, in plain terms . . . 11

7.2 Results . . . 11

7.3 The threshold is a clinical decision . . . 12

7.4 What this validation does and does not protect against . . . 12

**8. The real-time pipeline** 12

8.1 Ingestion gateway (services/ingestion-gateway) . . . 13

8.2 Kafka (docker-compose.yml) . . . 13

8.3 Preprocessing worker (services/worker/worker/main.py) . . . 13

8.4 Inference client and service . . . 13

8.5 Alerting . . . 14

8.6 Delivery guarantees and failure behavior . . . 14

8.7 Why these choices . . . 15

**9. Software testing and continuous integration** 15

**10. Scalability: the questions and how we answer them** **15**

10.1 Sizing the problem . . . 15
10.2 Measured: why batching matters . . . 16
10.3 The open questions . . . 17
10.4 One honest limit on latency . . . 17

**11. Safety, privacy and regulatory notes** **17**

**12. Limitations and roadmap** **18**

**Appendix A. Repository map** **18**

**Appendix B. Configuration reference** **18**

**Appendix C. Glossary** **19**

**Appendix D. References** **19**

# 1. Executive summary

NeuroStream is a research prototype that answers one question: **can a model trained on labeled EEG recordings flag seizure activity within one second of signal, inside a real-time system that keeps working when parts of it fail?** It has two halves that this guide explains in turn. The first is a **model half**: real EEG, preprocessing, a small convolutional neural network, and an evaluation designed to avoid flattering itself. The second is a **systems half**: an ingestion gateway, a Kafka queue, a preprocessing worker, a gRPC inference service and an alerting step.

What was built, and how each part was verified, in one table:

| Part | What it is | How it is verified |
|---|---|---|
| **Data and preprocessing** | University of Bonn EEG; sliding-window normalization and FFT band power, in one shared module | 7 unit tests; the same code runs in training and in serving |
| **Model and evaluation** | 1D CNN (17,858 parameters); recording-level split; threshold tuned on validation data | Seeded, reproducible script; test set scored once; metrics stored in models/meta.json |
| **Ingestion gateway** | NestJS service: validates input, publishes to Kafka keyed by patient | 14 automated tests including real HTTP requests against the actual app with a fake Kafka producer |
| **Streaming pipeline** | Kafka (KRaft), Python worker, gRPC inference, dead-letter topic | CI builds the full Docker stack on every push and replays held-out EEG end to end |
| **Failure handling** | Retries with backoff, offset rewind, dead-letter routing | Commit and rewind logic covered by 6 unit tests with fake Kafka and inference; **not** yet tested against a real broker or a killed container (see 8.6) |
| **Scalability** | Design reasoning and in-process micro-benchmarks | **Not load-tested.** Section 10 separates measured facts from estimates |

**Headline result.** On 1,725 one-second test epochs from recordings never used in training: accuracy 97.6%, sensitivity 95.1% (328 of 345 seizure epochs flagged), precision 93.2%, ROC AUC 0.998, with 24 false alarms among 1,380 seizure-free epochs.

**How to read that result honestly.** The Bonn benchmark is clean, single-channel and widely reported to be easy, and its recordings come from only a handful of people (section 3.4). These figures show that the method and the engineering work; they are not evidence that it would work in a hospital.

# 2. System overview

The system moves one second of EEG signal (an **epoch**) from a monitor to a decision. Every component is a separate process with a narrow job, so each can fail, restart and scale independently. The contract between components is a message, never shared memory.

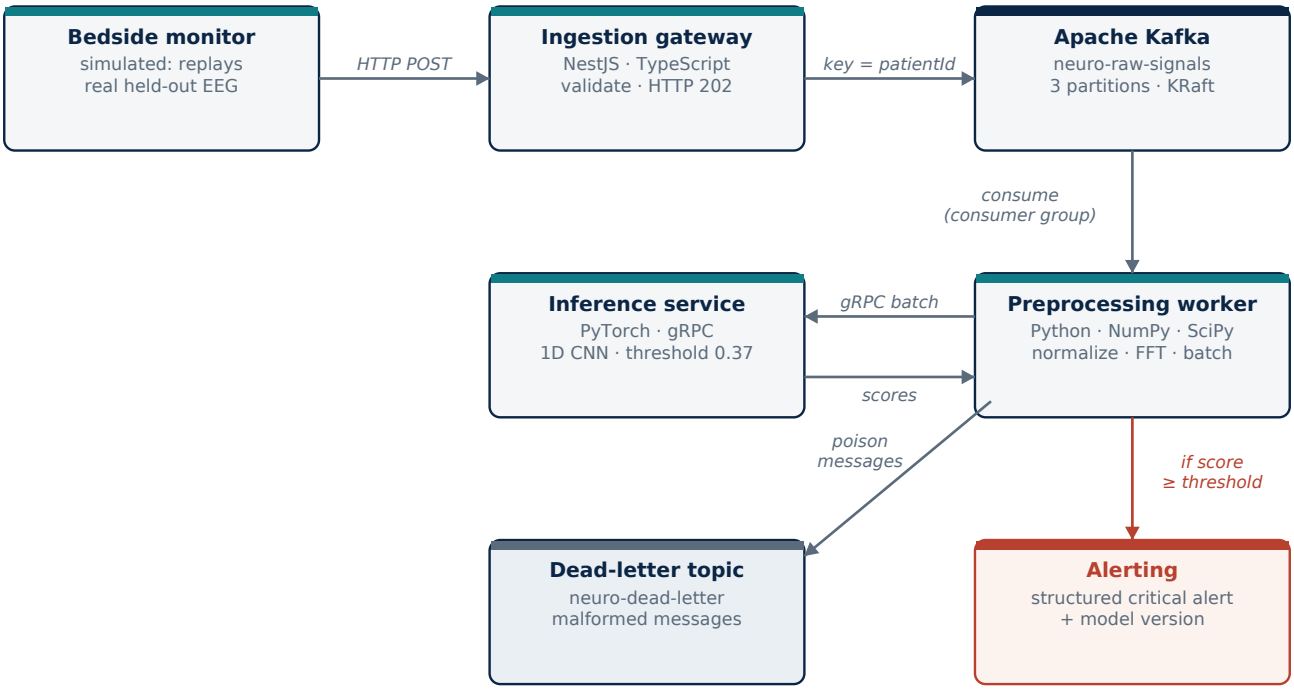



*Figure 2.1. Runtime architecture. Teal bars mark services we wrote; the navy bar marks infrastructure. The coral path is the alert.*

| Component | Technology | Responsibility | Key setting |
|---|---|---|---|
| **Ingestion gateway** | NestJS 10, TypeScript, class-validator | Accept POST `/api/v1/telemetry/stream`, validate, publish to Kafka, answer 202 | JSON limit 2 MB; Kafka retries 8; publish timeout 5 s |
| **Message queue** | Apache Kafka 7.6 (KRaft, single node) | Durable buffer between producers and consumers; per-patient ordering | 3 partitions; auto topic creation disabled |
| **Preprocessing worker** | Python, confluent-kafka, NumPy, SciPy | Consume, validate, normalize, compute band power, batch, call inference, raise alerts | Batch 8 or 500 ms; manual offset commit |
| **Inference service** | PyTorch, gRPC, Protocol Buffers | Score each epoch with the CNN, apply the tuned threshold | Batch limit 256; 4 workers; 2 torch threads |
| **Alerting** | Python module | Build a structured critical alert with the model version attached | Simulated: written to the log |

## 2.1 The three message formats

| Hop | Format | Fields |
|---|---|---|
| **Monitor → gateway** | HTTP POST, JSON | `patientId` (string, 1–64 chars), `timestamp` (positive number, epoch ms), `signalVector` (16–4,096 finite numbers) |
| **Gateway → Kafka** | JSON string, key = patientId | Exactly the validated payload. A typical 178-sample epoch is 1,020 bytes |
| **Worker → inference** | Protocol Buffers (proto/inference.proto) | `Sample`: patient_id, timestamp, signal (normalized), band_powers (5). Response: a `Prediction` per sample (anomaly_detected, confidence) plus model_version and server latency |

**Why two formats?** JSON at the edge is easy for any monitor vendor to produce and for humans to debug. Protocol Buffers between our own services gives a typed, versioned contract that is faster and fails loudly when the two sides disagree.

# 3. The data

## 3.1 What it is

The model is trained on the **University of Bonn EEG dataset** (Andrzejak et al., 2001, *Physical Review E* 64, 061907), a standard benchmark in seizure-detection research. It contains five sets of 100 single-channel recordings, each about 23.6 seconds long, sampled at 173.61 Hz (4,097 samples per file). The files are plain text, one number per line.

| Set | Recording | Condition | Label used here |
|---|---|---|---|
| **Z (A)** | Scalp EEG | Healthy volunteers, eyes open | seizure-free (0) |
| **O (B)** | Scalp EEG | Healthy volunteers, eyes closed | seizure-free (0) |
| **N (C)** | Intracranial EEG | Epilepsy patients, between seizures | seizure-free (0) |
| **F (D)** | Intracranial EEG | Epilepsy patients, between seizures (from the epileptogenic zone) | seizure-free (0) |
| **S (E)** | Intracranial EEG | Epilepsy patients, **during seizures** | seizure (1) |

The task is therefore binary: **seizure (set S, 100 recordings) versus seizure-free (the other four sets, 400 recordings)**. That is a 20% seizure share, which is far higher than in real monitoring, where seizures are rare (see 3.4).

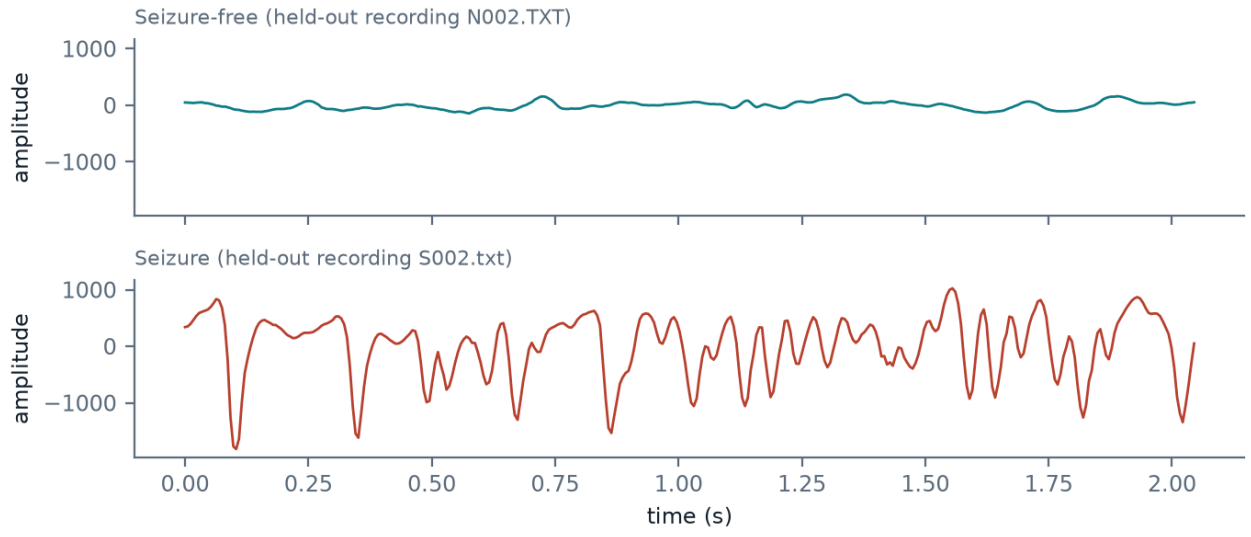



*Figure 3.1. Real held-out recordings, same vertical scale. A seizure appears as large, rhythmic, regular activity; seizure-free EEG is smaller and irregular.*

### 3.2 How it is stored and used

- **Source:** downloaded from the University of Bonn site with `training/download_data.py`; the full data is **not** committed.
- **In the repository:** only the 75 held-out *test* recordings (1.6 MB, `data/heldout/`) so the demo and CI can replay them. Attribution is included.
- **Epoching:** each recording is cut into one-second epochs of 178 samples. One epoch is the unit of every prediction.

### 3.3 Why this dataset

It is public, small enough to train on a laptop CPU in about a minute, and heavily reported in the literature, so results can be compared. Those same properties are its weakness (3.4).

### 3.4 Limitations of the data (important)

**The files carry no subject identifiers.** According to the original publication, as it is commonly summarized, the 500 recordings come from only about ten people: five healthy volunteers and five patients. We could not confirm this count from the data files themselves, which contain only numbers. If it is right, then **splitting by recording is not splitting by person**: recordings from the same individual appear in training and in the test set, so the test result is likely **optimistic about how the model would perform on a new patient**. A true patient-independent evaluation needs a dataset with patient IDs (for example CHB-MIT), and that is the first item on the roadmap.

- **Single channel.** Clinical EEG is typically 19–256 channels; the model sees one at a time.
- **Clean, short and selected.** Artifacts (muscle, eye blinks, electrode pops) that dominate real recordings were screened out.
- **Unrealistic prevalence.** 20% seizure epochs versus a small fraction of a percent in routine monitoring. At low prevalence the same false-alarm rate produces far more false alerts than true ones.

# 4. Preprocessing

Preprocessing lives in one module, `services/worker/worker/preprocessing.py`. The training script and the live worker import it, so the model is always trained on exactly what it will see in production. Mismatched preprocessing between training and serving (*train/serve skew*) is a classic silent failure.

### 4.1 Step 1: validation

An epoch must be a non-empty one-dimensional array of finite numbers. Anything else raises an error; in the worker that sends the message to the dead-letter topic.

### 4.2 Step 2: sliding-window normalization

Different recordings have different baselines and amplitudes. For each sample the code subtracts a local mean and divides by a local standard deviation, computed over a centered 64-sample window (0.37 s):

```
z[i] = (x[i] − mean_64(i)) / sqrt(var_64(i) + 1e-8)
```

Both statistics come from `scipy.ndimage.uniform_filter1d` (reflect mode at the edges), so the whole epoch is processed in one vectorized pass. The window is clamped to the epoch length.
**Trade-off:** a local window removes absolute amplitude, one of the signs of a seizure. The model recovers most of the information from waveform shape and from the band powers below, but amplitude is genuinely discarded.

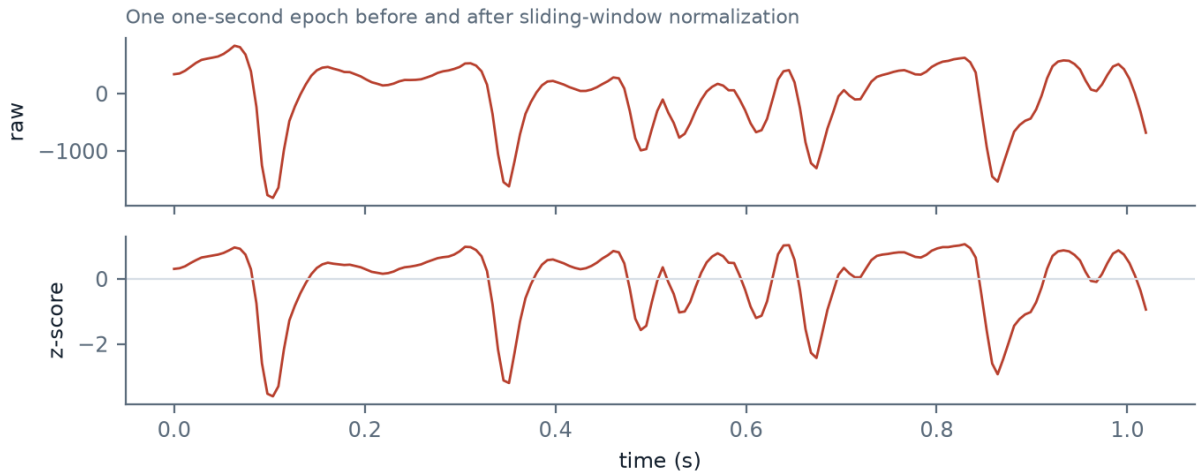



*Figure 4.1. Normalization removes baseline drift and scale while preserving the waveform's shape.*

## 4.3 Step 3: band power by FFT

A Hann window is applied to the mean-removed epoch, a real FFT gives the power spectrum, and the power inside each clinical band is divided by the total power, giving five **relative band powers** that sum to about 1:

| Band | Frequency | Clinical association (general) |
|---|---|---|
| **delta** | 0.5–4 Hz | Slow activity; deep sleep; also pathological slowing |
| **theta** | 4–8 Hz | Drowsiness; in seizures, often prominent rhythmic activity |
| **alpha** | 8–13 Hz | Relaxed wakefulness, eyes closed |
| **beta** | 13–30 Hz | Active thinking; also medication effects |
| **gamma** | 30–100 Hz | Fast activity; very little energy in this dataset |

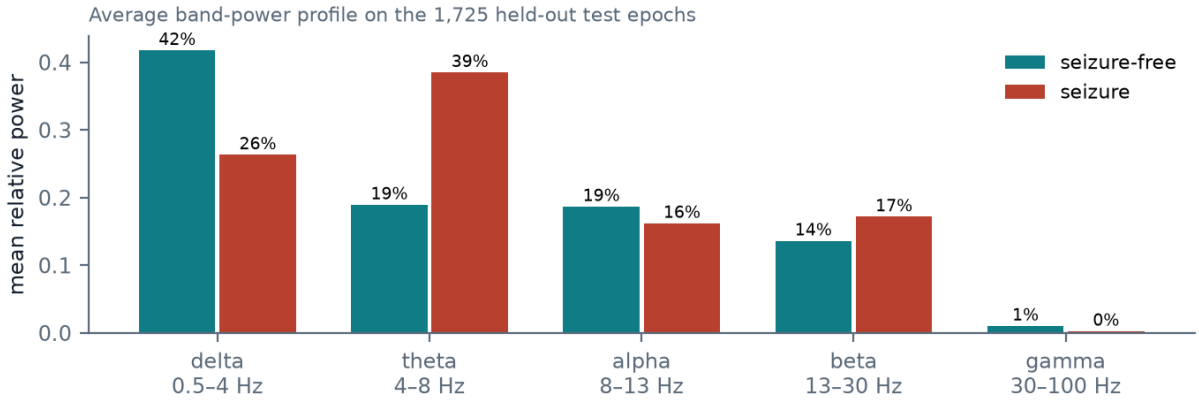



*Figure 4.2. In the held-out data, seizure epochs shift power from delta toward theta on average. The difference is real but modest, which is why the model also reads the waveform itself.*

**Cost.** Measured in-process on the development laptop: preprocessing one epoch takes about **0.14 ms** (median of 400 runs).

# 5. The model

## 5.1 Architecture

A small one-dimensional convolutional neural network (CNN). Convolutions slide short filters along the waveform and learn patterns such as sharp rises or rhythmic bursts; stacking three layers lets later filters respond to longer, more abstract patterns. The five band powers are appended before the final decision layers.

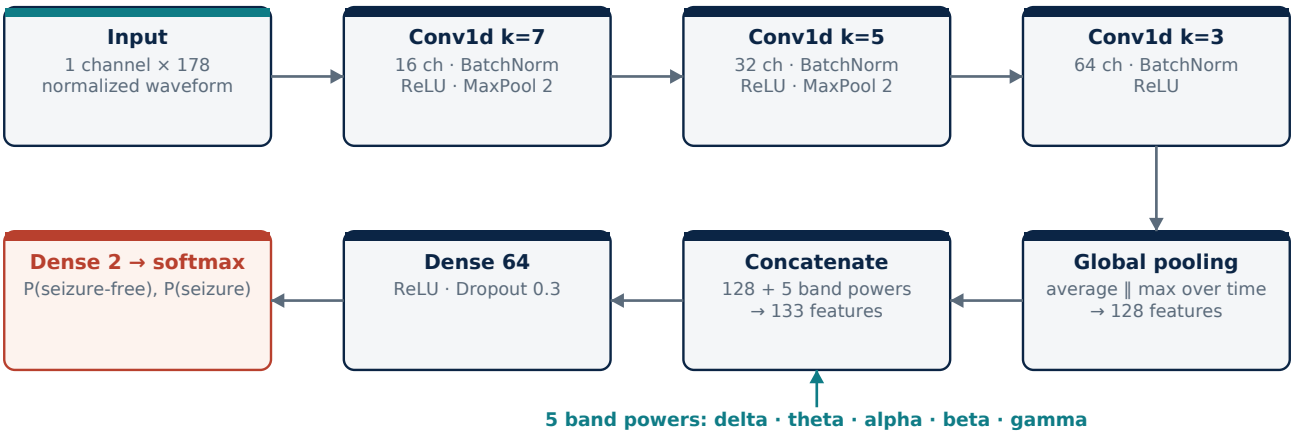



*Figure 5.1. Model architecture: 17,858 trainable parameters in total.*

| Layer | Output shape | Parameters |
|---|---|---|
| **Conv1d (k=7) + BatchNorm + ReLU + MaxPool 2** | 16 × 89 | 160 |
| **Conv1d (k=5) + BatchNorm + ReLU + MaxPool 2** | 32 × 44 | 2,656 |
| **Conv1d (k=3) + BatchNorm + ReLU** | 64 × 44 | 6,336 |
| **Global average pool ‖ global max pool** | 128 | 0 |
| **Concatenate 5 band powers** | 133 | 0 |
| **Dense 64 + ReLU + Dropout 0.3** | 64 | 8,576 |
| **Dense 2 (logits) → softmax** | 2 | 130 |

## 5.2 Design decisions

- **Why a small CNN, not a large model?** 17,858 parameters train in about a minute on a CPU, serve at a fraction of a millisecond per epoch, and are far less prone to memorizing 350 training recordings.
- **Why 1D, not a spectrogram image?** The signal is a time series. A first prototype used a pre-trained image network on stretched waveforms; it was replaced because its head was untrained and an image backbone is the wrong inductive bias here.
- **Why global average *and* max pooling?** Average captures sustained activity; max captures brief, sharp events. Seizure patterns can be either.
- **Why append band powers?** They give the classifier an explicit, interpretable spectral summary that is cheap to compute and that clinicians recognize.
- **Output.** A softmax probability P(seizure). The service flags an epoch when P(seizure) ≥ **0.37** (section 7.3).

The architecture lives in `services/ml-server/app/model.py`, shared by the training script and the serving code, together with the loader that reads `models/spike_cnn.pt` and `models/meta.json` (version, input length, threshold and metrics).

# 6. Training

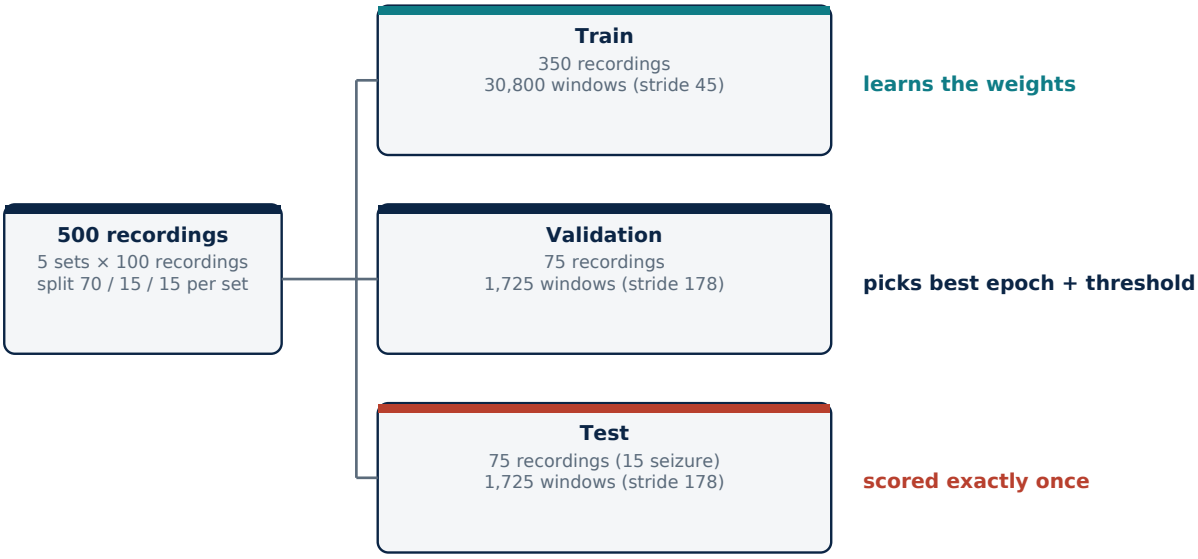



*Figure 6.1. The data split. Splitting is per recording, within each of the five sets, so every set is represented in every split.*

## 6.1 Procedure

| Step | Detail |
|---|---|
| **Split** | Within each set, a seeded random permutation assigns 70 recordings to train, 15 to validation, 15 to test (350 / 75 / 75). Seed 42. |
| **Windows** | Train: overlapping epochs with stride 45 samples (≈ 4× augmentation) → 30,800 windows, 20% seizure. Validation and test: non-overlapping epochs, 23 per recording → 1,725 each. |
| **Features** | Each window goes through the shared preprocessing: normalized waveform (178) and band powers (5). |
| **Loss** | Cross-entropy with the seizure class weighted ×4.0 (the inverse class ratio), so the rarer class is not ignored. |
| **Optimizer** | Adam, learning rate 2e-3, weight decay 1e-4, batch size 128, 30 epochs. |
| **Model selection** | After every epoch the model is scored on the validation set; the weights from the best validation ROC AUC are kept (epoch 24, AUC 0.9994). |
| **Threshold** | Chosen on **validation** data by maximizing F1 over 0.05–0.95 in steps of 0.01 → **0.37**. The test set plays no part. |
| **Test** | Scored exactly once, at the end, with the frozen weights and threshold. |

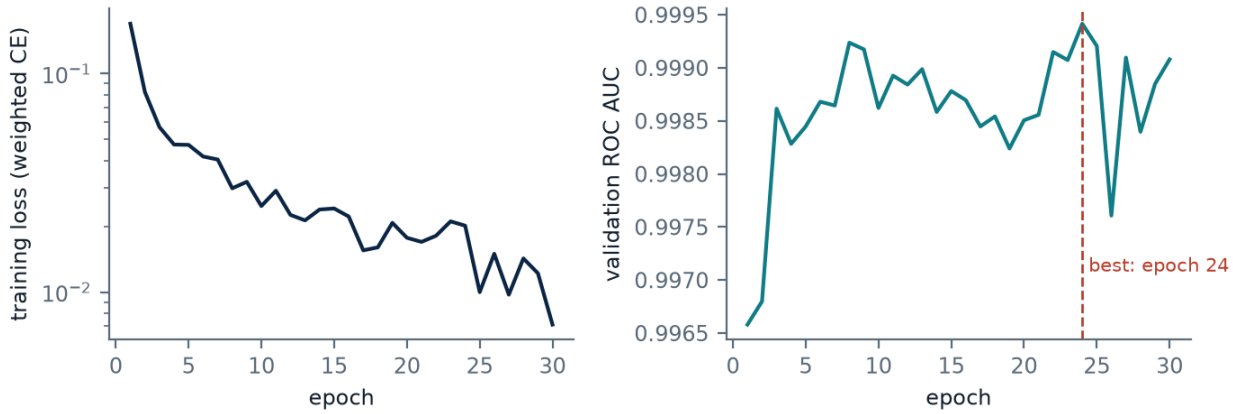



*Figure 6.2. Left: the training loss falls steadily. Right: validation AUC is high from the first epoch and plateaus, which is a sign the task is easy, not that tuning was heroic.*

## 6.2 Why overlapping windows only for training

Overlap multiplies the number of training examples and teaches the model that a seizure pattern can appear at any position within the epoch. Validation and test epochs do not overlap, so no two evaluation examples share samples and the metrics are not inflated by near-duplicates.

## 6.3 Reproducibility

- Fixed seed (42) for the split, the weight initialization and the shuffling; re-running `training/train.py` reproduces the reported metrics exactly.
- `docs/make_training_log.py` replays the same loop and saves the curves above without touching the saved model.
- The trained weights (under 100 KB) and the metrics are committed, so the pipeline and CI do not need to retrain.

# 7. Testing and validation of the model

## 7.1 The metrics, in plain terms

| Metric | Question it answers | Why a clinician cares |
|---|---|---|
| **Sensitivity (recall)** | Of the seizure epochs, how many did we flag? | Missed seizures are the costly error |
| **Precision (PPV)** | Of the epochs we flagged, how many were real? | Low precision means alarm fatigue |
| **Accuracy** | How often is the label right overall? | Misleading when classes are imbalanced |
| **ROC AUC** | How well do scores separate the two classes at *every* threshold? | Threshold-independent quality of the scores |

## 7.2 Results

| | Accuracy | Sensitivity | Precision | ROC AUC | TP | FP | FN | TN |
|---|---|---|---|---|---|---|---|---|
| **Validation (75 recordings)** | 99.1% | 97.4% | 98.2% | 0.9994 | 336 | 6 | 9 | 1374 |
| **Test (75 recordings)** | **97.6%** | **95.1%** | **93.2%** | **0.9978** | 328 | 24 | 17 | 1356 |

TP/FP/FN/TN are true positives, false positives, false negatives and true negatives at the deployed threshold. Test performance is somewhat lower than validation, as expected: the threshold was tuned on validation data.

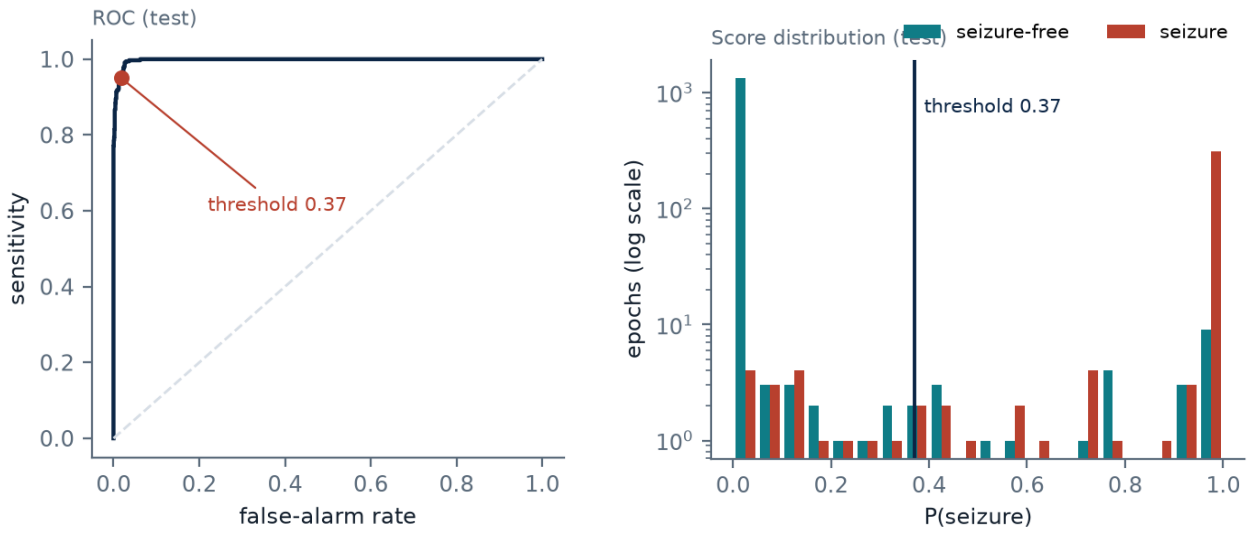



*Figure 7.1. Left: ROC curve on the test set with the deployed operating point. Right: the model's scores are strongly bimodal: most epochs score near 0 or near 1, and the errors sit in between.*

## 7.3 The threshold is a clinical decision

Every setting trades missed seizures against false alarms. Moving the line right reduces false alarms and misses more seizures; moving it left does the reverse. The deployed value (0.37) maximizes F1 on validation data, which weights both errors equally. A real deployment might weight a missed seizure far more heavily.

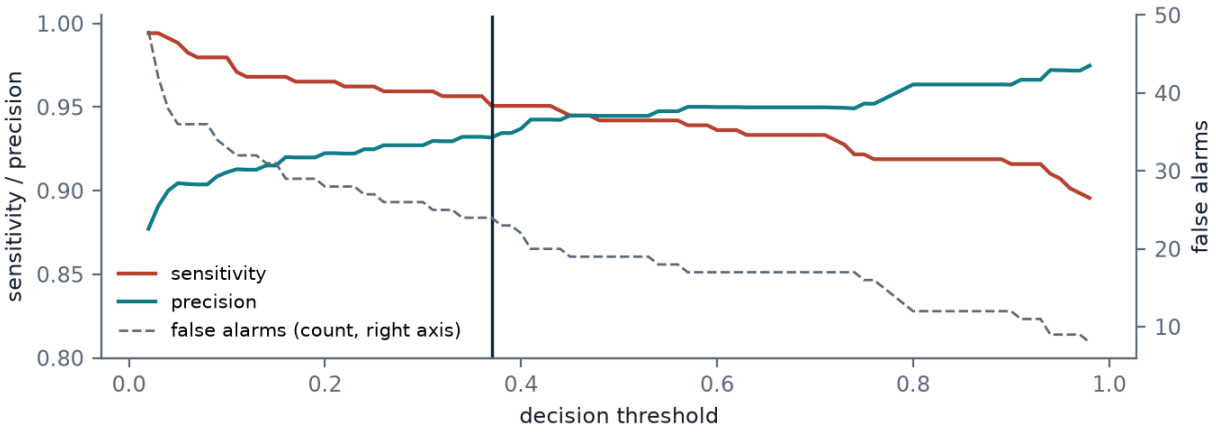



*Figure 7.2. Sensitivity, precision and false-alarm count against the threshold on the test set. Shown for illustration only: choosing a threshold on test data would overstate performance.*

| Threshold | Sensitivity | Precision | Missed seizure epochs | False alarms |
|---|---|---|---|---|
| **0.10** | 98.0% | 91.1% | 7 of 345 | 33 of 1,380 |
| **0.25** | 96.2% | 92.5% | 13 of 345 | 27 of 1,380 |
| **0.37 (deployed)** | 95.1% | 93.2% | 17 of 345 | 24 of 1,380 |
| **0.50** | 94.2% | 94.5% | 20 of 345 | 19 of 1,380 |
| **0.75** | 92.2% | 95.2% | 27 of 345 | 16 of 1,380 |
| **0.90** | 91.6% | 96.3% | 29 of 345 | 12 of 1,380 |

## 7.4 What this validation does and does not protect against

| Risk | Status |
|---|---|
| **Test data used for training or tuning** | **Guarded.** Weights fit on train; epoch and threshold chosen on validation; test scored once. |
| **Near-duplicate epochs across splits** | **Guarded.** Evaluation epochs do not overlap; splits are by recording. |
| **Same *person* in train and test** | **Not guarded.** Files carry no subject IDs (3.4). Likely optimistic for new patients. |
| **Class prevalence unlike the real world** | **Not addressed.** 20% seizure epochs here; precision would be far lower at realistic prevalence. |
| **Multichannel, artifacts, other hardware** | **Not addressed.** Single clean channel only. |
| **Calibration of the probabilities** | **Not evaluated.** The score ranks epochs well; it is not shown to be a calibrated probability. |
| **Seizure-level (event) metrics** | **Not evaluated.** Everything is per one-second epoch; clinicians reason in seizure events and false alarms per hour. |

# 8. The real-time pipeline

This chapter follows one epoch from the monitor to the alert and explains what each component guarantees. File paths are relative to the repository root.

## 8.1 Ingestion gateway (services/ingestion-gateway)

- **Endpoint:** `POST /api/v1/telemetry/stream` returns **202 Accepted** (the message is queued, not yet analyzed) with {accepted, patientId, samples}. `GET /health` for liveness.
- **Validation** (class-validator, global pipe with `whitelist` and `forbidNonWhitelisted`): patientId 1–64 chars; timestamp a positive finite number; 16–4,096 finite numbers. Unknown fields are rejected. Invalid input returns 400 and **nothing is published**.
- **Publish:** the validated JSON is emitted to `neuro-raw-signals` with **key = patientId**, so all epochs of a patient land on one partition and keep their order. The call has a 5 s timeout.
- **Failure:** if Kafka is unreachable the client gets **503** with a generic message. A global exception filter ensures stack traces and broker addresses never leak. The producer retries 8 times with backoff first.
- **Shutdown:** shutdown hooks flush and close the producer on SIGTERM.

## 8.2 Kafka (docker-compose.yml)

| Item | Setting | Why |
|---|---|---|
| **Mode** | Single node, KRaft (no ZooKeeper) | Light enough for a laptop; one process to run |
| **neuro-raw-signals** | 3 partitions, replication 1 | 3 partitions allow up to 3 parallel consumers; replication 1 is a laptop compromise |
| **neuro-dead-letter** | 1 partition | Low volume; holds messages a human should inspect |
| **Topic creation** | Automatic creation disabled; a one-shot init job creates the topics | A typo in a topic name fails loudly instead of silently creating a new topic |

## 8.3 Preprocessing worker (services/worker/worker/main.py)

The worker is a single loop built around `confluent-kafka` with **auto-commit disabled**. Offsets are committed by hand, only after a message has been fully handled.

```
loop:
  msg = consumer.poll(0.1)
  if msg:        ingest(msg)          # parse, validate, preprocess, add to batcher
  if batcher.should_flush():    flush()   # size >= 8  OR  oldest item >= 500 ms

flush():
  response = grpc.PredictBatch(batch)     # retries + backoff inside the client
  for each prediction: log; if anomaly -> dispatch_alert()
  commit offsets (max offset + 1 per partition)
```

- **Dynamic batching.** A batch is sent when it reaches 8 epochs or when its oldest epoch is 500 ms old, whichever comes first. Larger batches use the CPU more efficiently (section 10.2); the age limit bounds the added latency.
- **Poison messages.** If JSON is invalid, a field is missing, or the signal is shorter than 16 samples, the message is copied to `neuro-dead-letter` with `error` and `source` headers, then skipped. One bad record cannot block the stream.
- **Graceful shutdown.** On SIGTERM the loop stops, flushes the pending batch, flushes the dead-letter producer and closes the consumer.

## 8.4 Inference client and service

- **Retries.** The gRPC client retries on UNAVAILABLE, DEADLINE_EXCEEDED and RESOURCE_EXHAUSTED up to 4 times with exponential backoff (0.25 s × $2^{\text{attempt}}$, capped at 8 s) and random jitter, to avoid synchronized retry storms.
- **Server validation.** Empty batch, more than 256 epochs, or any signal shorter than 16 samples returns INVALID_ARGUMENT; unexpected exceptions return INTERNAL.

- **Server work.** Stack the epochs, run the CNN once for the whole batch, apply softmax, flag epochs with P(seizure) ≥ the threshold stored beside the weights. The reply carries the model version and the server-side latency.
- **Model versioning.** Every response and every alert carries the model version, so any alert can be traced to the exact model that raised it.

## 8.5 Alerting

An alert is a structured record: type `CRITICAL_NEURO_ANOMALY`, severity, patientId, the epoch timestamp, the confidence, the model version, a UTC time and a recommended action. Here it is written to the log; in production this step would call a paging or notification service. That is the step that closes the loop for the clinician.

## 8.6 Delivery guarantees and failure behavior

The pipeline aims for **at-least-once** delivery: an epoch may be processed more than once after a crash, but it should not be lost. The table states what the code does in each situation.

| Situation | Behavior | Verified by tests? |
|---|---|---|
| **Gateway receives invalid input** | 400, nothing published | Yes (6 cases in the HTTP test suite) |
| **Kafka down at the gateway** | Producer retries; then 503 without internals | Yes (fake broker failure) |
| **Malformed message in the topic** | Copied to the dead-letter topic with headers, then committed past | Yes (unit test) |
| **Malformed message arrives while valid ones are buffered** | Buffered messages are processed first; the malformed one is committed only after. If that processing fails, nothing is committed and all of them are redelivered | Yes (2 unit tests) |
| **Inference service briefly unavailable** | Client retries with backoff and jitter | No |
| **Inference unavailable after all retries** | Batch not committed; consumer seeks back to the earliest failed offset and tries again | Yes (unit test, fake client) |
| **Inference rejects the batch (non-retryable)** | Whole batch sent to the dead-letter topic, then committed | Yes (unit test) |
| **Worker crashes after inference, before commit** | Epochs are re-delivered and processed again: **duplicate alerts are possible** | No |
| **Worker receives SIGTERM** | Flushes the pending batch, then exits | No |

### A defect found while writing this guide, now fixed

**Poison commit could overtake unprocessed messages.** When a malformed message was dead-lettered, its offset was committed immediately. Kafka offsets are cumulative, so earlier valid messages from the same partition that were still waiting in the batch buffer were implicitly marked as done; a crash before the next flush (a window of up to 500 ms) would have lost them. **Fix:** before a malformed message is committed the worker now flushes the buffered batch, and commits only if that succeeds. If inference is unavailable, nothing is committed, and the buffered messages *and* the malformed one (even on another partition) are rewound and redelivered. A regression test fails on the old behavior and passes on the new.

### Gaps that remain open

- **No duplicate suppression.** At-least-once delivery can raise the same alert twice. *Fix:* deduplicate on (patientId, timestamp) at the alerting step.
- **No rebalance handling.** There is no partition-revocation callback to flush or commit before ownership moves. This matters as soon as more than one worker runs.

- **Failure paths are tested only with fakes.** The commit and rewind logic is unit-tested, and CI covers the happy path against real Kafka. A resilience test (stop the inference container mid-stream, check nothing is lost) is the natural next test.

### 8.7 Why these choices

- **Kafka between the gateway and the worker** so a slow or restarting worker never blocks, and never loses, incoming monitoring data; it also lets the worker be scaled by adding consumers.
- **Key by patient** so one patient's epochs stay in order, which matters for any future logic that looks across consecutive epochs.
- **gRPC to the model** so the model can be deployed, scaled and versioned independently of the worker.
- **Manual offset commit** because commit-after-success is the simplest way to avoid silent loss.

## 9. Software testing and continuous integration

| Level | What | Count / scope |
|---|---|---|
| **Unit (worker)** | Normalization finds a spike; band power peaks at the right band for a 10 Hz sine; invalid input is rejected; batcher flushes on size and on age. Offset-commit safety with fake Kafka and inference: a malformed message never lets a commit overtake buffered messages, failed inference rewinds without committing, non-retryable errors are dead-lettered | 13 tests |
| **Unit (gateway)** | DTO validation rules | 5 tests |
| **HTTP end-to-end (gateway)** | The real NestJS app over real HTTP with an in-memory fake Kafka: valid chunk → 202 and exact payload keyed by patient; six invalid cases → 400 and nothing published; broker down → 503; health | 9 tests |
| **Full-stack end-to-end (CI)** | Docker Compose builds Kafka, gateway, worker and inference; the script posts 12 held-out seizure and 12 held-out seizure-free epochs; passes only if ≥ 9 seizure epochs raise alerts, ≤ 1 false alarm, and every alert carries a model version | Every push |

**What CI caught.** The first full-stack run failed, three times, for reasons unit tests could not see: (1) the inference container crashed at startup because a model path was computed with a parent-directory lookup that does not exist inside the container; (2) the gateway image failed to build because a type-definitions package was missing; (3) the Python images could not be built because a pinned protobuf version conflicted with the gRPC tools. All three were fixed, and the run then passed. This is the strongest argument for testing the assembled system, not only its parts.

## 10. Scalability: the questions and how we answer them

The prototype runs on one laptop. This chapter lists the questions that must be answered before it could serve many patients, says what is **known** (measured here), what is **designed but not built**, and what is **still open**. Nothing in this chapter has been load-tested.

### 10.1 Sizing the problem

The unit of work is one epoch per channel per second, so the message rate equals the number of monitored channels. With the measured message size of 1,020 bytes:

| Monitored channels | Messages per second | Ingress (JSON) | Compute at measured per-epoch cost* |
|---|---|---|---|
| **100** | 100 | 0.10 MB/s | 0.02 cores |
| **1,000** | 1,000 | 1.02 MB/s | 0.24 cores |

| Monitored channels | Messages per second | Ingress (JSON) | Compute at measured per-epoch cost* |
|---|---|---|---|
| **10,000** | 10,000 | 10.2 MB/s | 2.4 cores |
| **100,000** | 100,000 | 102 MB/s | 24 cores |

*Preprocessing 0.14 ms + model scoring 0.096 ms per epoch at batch 32, measured in-process on an 8-core Windows laptop CPU with 2 torch threads. This is pure compute. It excludes JSON parsing, Kafka, gRPC serialization and network, which will dominate in practice, so treat the right-hand column as a floor, not a forecast.

## 10.2 Measured: why batching matters

| Epochs per model call | Median time per call | Time per epoch | Speed-up vs 1 |
|---|---|---|---|
| **1** | 0.73 ms | 0.732 ms | 1.0× |
| **8 (current default)** | 1.92 ms | 0.240 ms | 3.1× |
| **32** | 3.09 ms | 0.096 ms | 7.6× |
| **128** | 11.40 ms | 0.089 ms | 8.2× |

Per-call overhead dominates at small batch sizes, which is why the worker batches. The gain flattens beyond about 32 epochs, so the 8-epoch default leaves room to raise it under load.

## 10.3 The open questions

| Question | Why it matters | Today | How we would answer it |
|---|---|---|---|
| **How many partitions?** | Partitions cap consumer parallelism (consumers ≤ partitions) and are painful to change once keys are in use | 3 | Size for target peak plus headroom (e.g. 12–48), decided before go-live; key stays patientId |
| **How do workers scale?** | One worker is a single Python process | 1 worker, no rebalance handling | Run N workers in one consumer group; add rebalance callbacks (8.6); autoscale on consumer lag |
| **What if the model is the bottleneck?** | Inference must keep up with the batches | Stateless gRPC server; 4 threads | Run replicas behind a gRPC-aware load balancer; larger dynamic batches; export to ONNX or TorchScript; quantize; GPU only if the model grows |
| **What happens during a burst or an outage?** | Monitoring data does not pause | Kafka buffers; gRPC retries | Retention sized for the longest outage we must survive; alert on consumer lag; deadlines so slow calls fail fast |
| **Can alerts be duplicated or lost?** | Duplicate pages cause fatigue; lost alerts are unsafe | At-least-once; the poison-commit defect is fixed (8.6); duplicates still possible | Deduplicate on (patientId, timestamp); consider a transactional outbox for alerts |
| **What is the end-to-end latency budget?** | The point is early warning | Not measured end to end | Define an objective (e.g. epoch captured → alert dispatched under N seconds); budget each hop; record it with tracing |
| **How do we run it with several channels per patient?** | Real EEG has many channels; message size and model input change | One channel per message | A channel-aware message and model; partition by patient so channels of one patient stay together |
| **How do we operate it?** | You cannot fix what you cannot see | Logs only | Metrics (lag, latency percentiles, alert and dead-letter rates), distributed tracing with a correlation ID, dashboards and on-call alerts on the system itself |
| **How do we change the model safely?** | A bad model is a patient-safety issue | Version string in every response; model baked into the image | Shadow mode, canary release, rollback; monitor score distribution drift; revalidate on every retrain |
| **How do we protect patient data?** | EEG tied to a patient is health data | Plain HTTP and plaintext Kafka; no auth | TLS everywhere, authentication and authorization, pseudonymous IDs, encryption at rest, audit logs, retention policy, data-protection review |

## 10.4 One honest limit on latency

A one-second epoch means the earliest possible alert comes one second after the seizure pattern begins, plus the pipeline. The configured 500 ms batch wait adds up to half a second at low load. Overlapping windows (for example one prediction every 250 ms) would cut detection delay at four times the compute, which is a trade-off worth measuring.

# 11. Safety, privacy and regulatory notes

None of the following is implemented or claimed. It lists what would have to be addressed before any clinical use.

- **Intended use and risk.** Software that informs clinical decisions is regulated as a medical device in most jurisdictions. It needs a defined intended use, risk analysis, documented validation and post-market monitoring.

- **Clinical validation.** Requires prospective data from the target setting, patient-independent evaluation, event-level metrics and false alarms per hour.
- **Privacy.** Patient-linked EEG is personal health data. Authentication, encryption in transit and at rest, access control, auditing and a retention policy are required.
- **Human oversight.** The design keeps a clinician in the loop: the system flags an epoch, and the clinician confirms it.

# 12. Limitations and roadmap

| Priority | Next step | What it would show |
|---|---|---|
| **1** | Train and evaluate on CHB-MIT (pediatric patients, 23 patients across 24 recording cases, multichannel); split by patient | Whether the result holds for a new patient: the biggest open question |
| **2** | Report seizure-event metrics and false alarms per hour; evaluate at realistic prevalence | Clinically meaningful performance |
| **3** | Add alert de-duplication and rebalance handling | A delivery guarantee that holds under failure |
| **4** | Resilience test in CI: stop the inference container mid-stream, check nothing is lost | Fault tolerance demonstrated, not just designed |
| **5** | Load test (k6 or a custom producer) and publish measured throughput and latency percentiles | Real capacity numbers |
| **6** | Metrics and tracing; calibration; model monitoring | Operability and trust in the score |

# Appendix A. Repository map

```
proto/inference.proto            gRPC contract (Sample, Prediction)
docker-compose.yml               Kafka (KRaft), topic init, ml-server, gateway, worker
services/ingestion-gateway/      NestJS gateway: DTO validation, Kafka producer, 14 tests
services/worker/worker/          preprocessing, batcher, inference_client, alerting, main
services/ml-server/app/          model.py (CNN + loader)  server.py (gRPC)
models/                          spike_cnn.pt  meta.json  heldout_test_files.json
training/                        train.py  download_data.py
data/heldout/                    75 held-out test recordings (attribution included)
scripts/e2e_check.py             full-stack check used by CI
demo/                            no-Docker demo and website data build
site/                            static website
.github/workflows/ci.yml         unit tests + Docker end-to-end
```

# Appendix B. Configuration reference

| Setting | Default | Effect |
|---|---|---|
| **KAFKA_BROKERS / KAFKA_TOPIC / KAFKA_DLQ_TOPIC** | kafka:9092 · neuro-raw-signals · neuro-dead-letter | Where the worker reads and dead-letters |
| **KAFKA_GROUP_ID** | neuro-preprocessing-workers | Consumer group; workers sharing it split the partitions |
| **BATCH_MAX_SIZE / BATCH_MAX_WAIT_MS** | 8 · 500 | Batch flushes at whichever limit is reached first |
| **SAMPLING_RATE_HZ / NORM_WINDOW** | 173.61 · 64 | Must match training for the band powers and normalization |
| **GRPC_TIMEOUT_S / GRPC_MAX_RETRIES** | 5 · 4 | Per-call deadline and retry budget |
| **MODEL_DIR / GRPC_PORT / MAX_WORKERS / TORCH_NUM_THREADS** | /srv/models · 50051 · 4 · 2 | Inference service |

# Appendix C. Glossary

| Term | Meaning |
|---|---|
| **EEG** | Electroencephalogram: voltage recorded from electrodes on or in the head |
| **Epoch** | A fixed-length slice of signal; here one second (178 samples) |
| **Seizure** | A burst of abnormal, synchronous neuronal activity |
| **Interictal** | Between seizures |
| **Z-score** | A value rescaled to 'how many standard deviations from the local mean' |
| **FFT** | Fast Fourier transform: splits a signal into its frequency components |
| **Band power** | Share of signal energy in a frequency range (delta, theta, alpha, beta, gamma) |
| **CNN** | Convolutional neural network: learns short patterns and combines them |
| **Held-out** | Data kept aside and never used to fit or tune the model |
| **Sensitivity / recall** | Fraction of true seizures that are flagged |
| **Precision / PPV** | Fraction of flags that are true seizures |
| **ROC AUC** | Probability that a random seizure epoch scores higher than a random seizure-free epoch |
| **Kafka** | A distributed log: producers append messages, consumers read them at their own pace |
| **Partition** | An ordered slice of a topic; the unit of parallelism |
| **Consumer group** | Consumers that share a topic's partitions between them |
| **Offset** | A message's position in a partition; committing records progress |
| **Dead-letter topic** | A side topic for messages that cannot be processed |
| **At-least-once** | A message is never lost but may be processed more than once |
| **gRPC / Protobuf** | A fast typed remote-call protocol and its schema language |

# Appendix D. References


- Andrzejak RG, Lehnertz K, Mormann F, Rieke C, David P, Elger CE (2001). Indications of nonlinear deterministic and finite-dimensional structures in time series of brain electrical activity: Dependence on recording region and brain state. *Physical Review E* 64, 061907.
- Dataset download page: University of Bonn, Department of Epileptology (Lehnertz group), 'Downloads'.
- Photo on the website: 'EEG Recording Cap' by Chris Hope, CC BY 2.0, via Wikimedia Commons.